

数据库学习笔记

Myisam和InnoDB区别

目 表格

	特性	InnoDB	MyISAM
1	事务支持	支持 (符合 ACID)	不支持
2	锁定级别	行级锁	表级锁
3	外键支持	支持	不支持
4	崩溃恢复	支持, 通过事务日志实现自动恢复	不支持, 崩溃可能导致数据损坏
5	性能优势	适合高并发、频繁写操作	适合大量读操作、少量写操作
6	数据存储	表数据和索引存储在同一个或独立文件中	表数据、索引和定义分别存储在 .frm, .m
7	缓存机制	缓存数据和索引	只缓存索引
8	全文索引	较新版本 (MySQL 5.6+) 支持	早期版本就支持

8 条记录

. 事务与数据完整性

- InnoDB:** 遵循 **ACID** (原子性、一致性、隔离性、持久性) 原则, 支持 **COMMIT** 和 **ROLLBACK** 操作。这保证了数据操作的可靠性, 特别适用于金融、电商等对数据完整性要求高的场景。
- MyISAM:** 不支持事务。写入操作一旦执行, 就无法回滚, 崩溃可能导致数据不一致。

1. 并发处理与锁定

- InnoDB:** 采用**行级锁**。当一个事务修改一行数据时, 只会锁定这一行, 其他事务可以同时读写其他行, 大大提高了高并发的性能。
- MyISAM:** 采用**表级锁**。任何读或写操作都会锁定整个表, 这在高并发场景下会导致严重的性能瓶颈。

2. 崩溃恢复与数据可靠性

- InnoDB:** 具有强大的**崩溃恢复能力**, 通过事务日志 (redo logs) 来确保数据库能在崩溃后自动恢复到之前的状态。

- **MyISAM**: 崩溃后数据可能会损坏，需要手动修复，恢复过程繁琐且不一定成功。

3. 性能考量

- **InnoDB**: 在**混合读写**或**写密集型**的应用中表现出色，因为其行级锁能有效处理并发。
- **MyISAM**: 结构简单，在**纯粹的读密集型**应用中（如数据仓库、报表系统）查询速度非常快。然而，随着并发写入的增加，其性能会迅速下降。

总结

在现代应用开发中，**InnoDB** 是绝大多数场景下的**首选**。它提供了数据完整性、高并发支持和强大的崩溃恢复能力，是构建可靠和可扩展系统的基础。而 **MyISAM** 仅在一些特殊场景下有其用武之地，比如只读或写操作非常少的应用。

一些基本sql语句语法使用

创建数据库和表

代码块

```
1  USE school;
2  DROP TABLES IF EXISTS `student`;
3  -- 创建表
4  CREATE TABLE IF NOT EXISTS `test`
5  (`id` VARCHAR(10) NOT NULL COMMENT '学号',
6   `name` VARCHAR(4) NOT NULL DEFAULT '张三' COMMENT '姓名',
7   `birthday` DATE COMMENT '生日',
8   `password` VARCHAR(12) DEFAULT '123456' ,
9   `email` VARCHAR(20) DEFAULT 'xxx@qq.com' COMMENT '邮箱',
10 PRIMARY KEY (`id`))
11 )ENGINE=INNODB DEFAULT CHARSET utf8;
12 -- 查看创建数据库的语句
13 SHOW CREATE DATABASE `school`;
14 -- 查看创建表的语句
15 SHOW CREATE TABLE `test`;
16 DESC `test`; -- 描述表的结构
```

修改表

主要关键字alter table

注意：改表名用rename，改字段名用change

改字段名时新名字后面应加上其数据类型

1. 修改表

```
2 ALTER TABLE `test` RENAME AS `test01`; -- 改表名
3 -- 增加字段
4 ALTER TABLE `test01` ADD address VARCHAR(20) DEFAULT 'xxx省市'
5 COMMENT '地址';
6 ALTER TABLE `test01` DROP `email`; -- 删除字段
7 ALTER TABLE `test01` MODIFY id INT(10) AUTO_INCREMENT
8 NOT NULL; -- 修改字段属性约束
9 ALTER TABLE `test01` CHANGE `id` `stu_id` INT(10); -- 字段重命名
```

数据管理

外键

(物理外键，一般其实不用)

语法：

1. 在创建表时使用：

(实际就是添加一个约束)

代码块

```
1 CREATE TABLE grade(
2 gradeid VARCHAR(10) COMMENT '学号',
3 gradename VARCHAR(4) NOT NULL COMMENT '名字',
4 PRIMARY KEY (gradeid)
5 ) ENGINE=INNODB DEFAULT CHARSET utf8;
6
7 CREATE TABLE IF NOT EXISTS `student`
8 (`id` VARCHAR(10) NOT NULL COMMENT '学号',
9 `name` VARCHAR(4) NOT NULL DEFAULT '张三' COMMENT '姓名',
10 `birthday` DATE COMMENT '生日',
11 `password` VARCHAR(12) DEFAULT '123456' ,
12 `email` VARCHAR(20) DEFAULT 'xxx@qq.com' COMMENT '邮箱',
13 PRIMARY KEY (`id`),
14 KEY FK_id(id),
15 CONSTRAINT FK_id FOREIGN KEY (id) REFERENCES grade (gradeid)
16 ) ENGINE=INNODB DEFAULT CHARSET utf8;
17
```

2. 创建后用alter来修改添加

代码块

```
1 ALTER TABLE student ADD CONSTRAINT FK_id FOREIGN (id) REFERENCES grade
```

```
(gradeid);
```

引用的称为从表，被引用的为主表，当外键存在时，主表不能直接被删除，需先删除从表才能删除主表。

DML语言（数据库操作语言）

insert语句：

插入一条数据：

Insert into 表名(字段1, 字段2, 字段3) values('值1','值2','值3'.....)

插入多条数据：

Insert into 表名(字段1, 字段2, 字段3) values('值1','值2','值3'.....), ('值1','值2','值3'.....), ('值1','值2','值3'.....).....

代码块

```
1  -- 一条
2  INSERT INTO student ( `id`,`name`,`birthday`) VALUES
3  ('12346457','李四','2025-01-01');
4  -- 多条
5  INSERT INTO student (`id`,`name`,`birthday`) VALUES ('1245673','zhsh','2004-
6  06-23'),
7  ('133456','李五','2005-08-08'),('133677','张三','2007-09-24');
```

字段名可省略，但是后面的值必须一一对应。

update语句：

语法：

Update 表名 set 字段 =值 where 条件

代码块

```
1  UPDATE `student` SET `name` = '大春' WHERE `name`='李五';
```

修改多个字段数据用逗号隔开

代码块

```
1  UPDATE `student` SET `password`='123444',`email`='234456@qq.com' WHERE
2  `name`='李四' AND `id`>'12345';
```

修改的值可以是变量（一般用于datetime类型）

代码块

```
1 UPDATE `student` SET `birthday`=CURRENT_TIME WHERE name='张三';
```

注意：不写where 条件会把所有数据改成设定的值

Where 子句:

Bool 类型，返回true或false

运算符:

=, <>或! =(不等) , <,<=,>,>=

Between and.....(在某个范围内)

连接多个子句:

AND (与, &&) , OR (或, ||)

delete或truncate(删除数据)

代码块

```
1 DELETE FROM `student` WHERE `name`='张三';
2 TRUNCATE `student`;
```

delete和truncate区别

都能清空数据，保留表结构

特性	DELETE	TRUNCATE TABLE
操作类型	DML (数据操纵语言)	DDL (数据定义语言)
执行速度	较慢（逐行删除）	极快（释放整个表空间）
回滚 (Rollback)	可以回滚	通常不可以回滚
WHERE 子句	可以使用，可实现条件删除	不可使用，删除所有数据
自增计数器	不重置 (除非手动操作)	重置为初始值（通常是 1）
触发器 (Trigger)	会触发 DML 触发器	不会触发 DML 触发器
日志记录	记录每一行删除操作，日志量大	记录释放表空间操作，日志量小
表结构	保留表结构	保留表结构

DQL（数据库查询语言）

查询表中所有信息：

代码块

```
1 SELECT *FROM student;
```

查询指定字段，可以as给字段起别名，也可给表起别名

代码块

```
1 SELECT `id` AS 学号 , `name` AS 姓名 FROM student;
```

函数concat (a,b),连接a,b

代码块

```
1 SELECT CONCAT('姓名: ', `name`) FROM `student`;
```

去重和表达式

数据库中表达式：文本值，表达式，列，null，系统变量，函数.....

代码块

```
1 -- 去除重复数据
2 SELECT DISTINCT `password` FROM `student`;
3 SELECT VERSION(); -- 查看数据库版本（函数）
4 SELECT 100*3-1 AS '式子' FROM `student`; -- 计算表达式
5 SELECT @@auto_increment_increment; -- 查看自增步长（变量）
6 SELECT `result`+1 FROM `student`; -- 把成绩加一后查看（不改变原表）
```

模糊查询

where子句运算符：

is null , is not null , between

like(a like b)看a,是否匹配b

In , a in (a1,a2,a3.....)看a是否存在于（）中的某个值，类似于a=a1||a=a2||a=a3.....

代码块

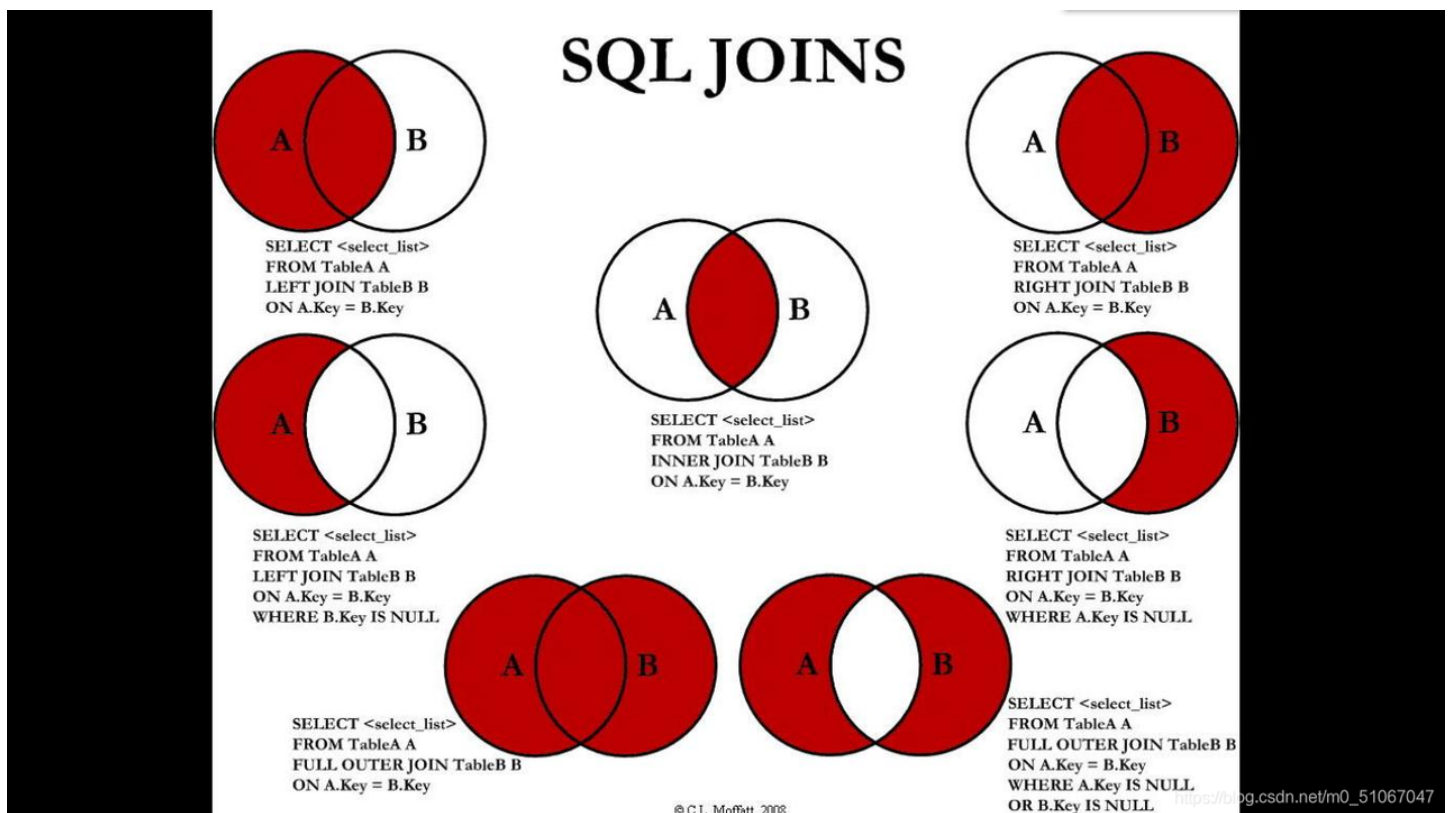
```
1 -- like结合%（代表0到任意个字符）_（一个字符），类似于正则表达式
2 -- 查询姓刘的同学
3 SELECT `name` FROM `student` WHERE `name` LIKE '刘%';
```

```

4  -- 查询姓刘且后面只带着1个字的
5  SELECT `name` FROM `student` WHERE `name` LIKE '刘_';
6  -- 查询姓刘且后面只带着2个字的
7  SELECT `name` FROM `student` WHERE `name` LIKE '刘__';
8  -- 查询名字中间带个成字的
9  SELECT `name` FROM `student` WHERE `name` LIKE '%成%';
10 -- in
11 -- 查询学号为(1234,12345,12346)这几名学生
12 SELECT `id`, `name`, `address`, `result` FROM `student` WHERE `id` IN
    (1234,12345,12346);
13 SELECT `id`, `name`, `address`, `result` FROM `student`
14 WHERE `id` =1234 OR `id` =12345 OR `id` =12346;

```

联表查询



代码块

```

1  -- 查询学号, 姓名, 科目, 成绩
2  SELECT s.id, `name`, `subject`, `score` FROM `result` AS r INNER JOIN `student` AS s
3  ON s.id=r.id;
4  -- 把右边的表全部信息都会打印出来, 也就是没有成绩的学生也会打印出来, subject,score输出是
    null
5  SELECT s.id, `name`, `subject`, `score` FROM `result` AS r RIGHT JOIN `student` AS s
6  ON s.id=r.id;
7  -- 查询学号, 姓名, 科目名称, 成绩 (多表查询)
8  SELECT s.id, s.`name`, sub.`name`, `score` FROM `result` AS r INNER
    JOIN `student` AS s

```

```
9 ON s.id=r.id INNER JOIN `subject` AS sub ON r.subject=sub.NO;
```

自联接

自连接（Self-Join）是 SQL 中一种非常强大的技巧，它允许一张表和他自身进行连接。虽然概念上是“自己连自己”，但在查询执行时，我们必须将这张表当作两张不同的表来对待。

通常，JOIN 是用来连接两个不同的表（例如 Employees 表和 Departments 表）。而自连接是指：将一张表与其自身的另一个副本进行连接，以在同一张表的行之间进行比较或组合。

要实现自连接，必须给这张表设置两个不同的别名（Alias），这样 SQL 引擎就知道你是在同时引用这张表的两个不同版本。

自连接主要用于处理表中的层级关系或同行比较问题，典型的应用场景包括：

1. **查找层级关系**：找出员工和他们的上级主管。
2. **同行比较**：找出工资比某人高或低的员工。
3. **递归查询**：查找组织架构中的所有下属（虽然在现代 SQL 中 CTE 递归更常见，但自连接是基础）。

自连接的语法与标准的 INNER JOIN 或 LEFT JOIN 非常相似，关键在于使用别名。

基本语法

SQL

代码块

```
1 SELECT
2     A.column1,
3     B.column2
4 FROM
5     YourTable AS A -- 表的第一个别名 JOIN
6     YourTable AS B -- 表的第二个别名 ON
7     A.JoinColumn = B.JoinColumn -- 连接条件 WHERE
8     ...
```

在上面的语法中：

- `YourTable AS A`：我们将原表看作“左表”或“第一个副本”。
- `YourTable AS B`：我们将原表看作“右表”或“第二个副本”。

如果不使用别名，SQL 引擎无法区分你引用的列是来自“第一个表”还是“第二个表”，从而导致错误。

典型应用案例：员工与上级（层级关系）

我们用一个经典的例子来理解自连接：找出每位员工及其对应的上级主管的姓名。

示例数据表： `Employees`

ID	Name	ManagerID
101	张三 (Zhang San)	104
102	李四 (Li Si)	104
103	王五 (Wang Wu)	105
104	赵六 (Zhao Liu)	NULL
105	钱七 (Qian Qi)	104

自连接 SQL 语句

在这个表中，一个员工的 `ManagerID` 实际上是其上级主管的 ID，而这个 ID 也可以在 `ID` 列中找到。
我们连接的条件就是：员工表的 `ManagerID` 等于主管表的 `ID`。

SQL

代码块

```
1  SELECT
2      E.Name AS EmployeeName,      -- E 代表员工 (Employee)
3      M.Name AS ManagerName        -- M 代表主管 (Manager)FROM
4      Employees AS E
5  INNER JOIN
6      Employees AS M                -- 将 Employees 表当作 MON
7      E.ManagerID = M.ID;           -- E 的上级ID 等于 M 的员工ID
```

查询结果

EmployeeName	ManagerName
张三	赵六
李四	赵六
王五	钱七
钱七	赵六

分页和排序

关键字：limit 和 order by

分页：limit 起始位置 ， 页面大小

```
1 SELECT s.id,s.name,sub.name,score FROM student AS s INNER JOIN result AS r
2 ON s.id=r.id INNER JOIN `subject` AS sub ON r.`subNO`=sub.NO
3 LIMIT 0,5;
```

假如页面大小为page_size

第n页可推出为limit (n-1)page_size,page_size

假设总数据量为total_size

则求解一共有几页：total_size/page_size向上取整、

排序：

代码块

```
1 SELECT s.id,s.name,sub.name,score FROM student AS s INNER JOIN result AS r
2 ON s.id=r.id INNER JOIN `subject` AS sub ON r.`subNO`=sub.NO
3 WHERE `subNO`=1 ORDER BY score ASC; -- desc降序,asc升序
```

子查询的分类与类型

子查询可以根据其返回的结果类型和与主查询的关系进行分类。

(1) 单行子查询 (Single-Row Subquery)

子查询只返回**一行**记录（可以是单列或多列）。

- **用途：** 常用在 `WHERE` 子句中，与单行比较运算符（如 `=`，`>`，`<`，`>=` 等）一起使用。
- **注意：** 如果子查询返回多行，查询将报错。

示例： 查找工资高于平均工资的员工。

SQL

代码块

```
1 SELECT employee_name, salary
2 FROM employees
3 WHERE salary > (SELECT AVG(salary) FROM employees); -- 子查询返回一个平均值
```

(2) 多行子查询 (Multiple-Row Subquery)

子查询返回**多行**记录。

- **用途：** 常用在 `WHERE` 或 `HAVING` 子句中，必须与多行比较运算符一起使用。

- **运算符：**

- **IN**：匹配列表中的**任何一个**值。
- **NOT IN**：不匹配列表中的**任何一个**值。
- **ANY / SOME**：匹配列表中的**任何一个**值（例如：> ANY 表示大于最小的那个值）。
- **ALL**：匹配列表中的**所有**值（例如：> ALL 表示大于最大的那个值）。

示例：查找在部门 'Sales' 或 'IT' 工作的员工。

SQL

代码块

```
1  SELECT employee_name
2  FROM employees
3  WHERE dept_id IN (SELECT dept_id FROM departments WHERE dept_name IN ('Sales',
    'IT'));
```

(3) 多列子查询 (Multiple-Column Subquery)

子查询返回**多列**记录。

- **用途：**在 **WHERE** 子句中，同时比较多列的值。

示例：查找与部门 10 中的某个员工具有相同 **job_id** 和 **salary** 的其他员工。

SQL

代码块

```
1  SELECT employee_name
2  FROM employees
3  WHERE (job_id, salary) IN (SELECT job_id, salary FROM employees WHERE dept_id
    = 10);
```

(1) 非相关子查询（独立子查询）

特点：子查询可以**独立执行**，不需要依赖外部查询的任何信息。

- **执行方式：**子查询先执行一次，得到结果后，将结果传递给外部查询。

(2) 相关子查询 (Correlated Subquery)

特点：子查询**不能独立执行**，它引用了主查询中的列。

- **执行方式：** 主查询取一行，子查询就执行一次，用于处理当前行的数据。这个过程会重复直到主查询的所有行都被处理。
- **性能：** 由于需要重复执行，相关子查询的性能通常低于非相关子查询（但并非绝对）。
- **关键词：** 最常见的形式是使用 `EXISTS` 或 `NOT EXISTS`。

示例（使用 EXISTS）： 查找至少有一个订单的客户。

SQL

代码块

```
1  SELECT customer_name
2  FROM customers c -- 主查询WHERE EXISTS (
3      SELECT 1 -- 子查询FROM orders o
4      WHERE o.customer_id = c.customer_id -- 子查询引用了主查询的列 c.customer_id
5  );
```

`EXISTS` 仅检查子查询是否返回任何行。如果返回行（即条件成立），`EXISTS` 返回 `TRUE`。

子查询的高级应用

1. 派生表（FROM 子句中的子查询）

- 当子查询出现在 `FROM` 子句中时，其结果集被视为一个临时的**派生表 (Derived Table)** 或**内联视图 (Inline View)**。
- **语法要点：** 派生表**必须**指定一个别名。
- **示例：** 查找每个部门的平均工资，然后再筛选出高于公司平均工资的部门平均工资。
- SQL

代码块

```
1  SELECT dept_id, avg_salary
2  FROM (
3      SELECT dept_id, AVG(salary) AS avg_salary -- 派生表 tFROM employees
4      GROUP BY dept_id
5  ) AS dept_avg
6  WHERE dept_avg.avg_salary > (SELECT AVG(salary) FROM employees);
```

在这个例子中，`dept_avg` 就是一个派生表。

2. 嵌套层次

子查询可以**多层嵌套**，即在一个子查询中再嵌套另一个子查询。虽然理论上没有严格限制，但过深的嵌套（如超过 3-4 层）会使 SQL 语句难以阅读和维护，并可能影响查询优化器的工作，降低性能。

3. 与 JOIN（连接）的对比

很多可以使用子查询解决的问题，也可以用 **JOIN**（特别是 **INNER JOIN** 或 **LEFT JOIN**）来解决。

特性	子查询	JOIN
可读性	复杂查询可能更清晰，尤其是在 WHERE 条件中。	涉及多表连接时结构清晰。
性能	非相关子查询 通常与 JOIN 性能相近。	对于大表连接，优化器通常能更好地优化 JOIN 语句。
适用性	适合作为 条件 （如 IN, EXISTS）。	适合 合并数据 ，显示多表中的列。

常用 MySQL 函数概述

1. 字符串函数 (String Functions)

这些函数用于处理和操作字符串数据。

函数名	说明	示例
LENGTH(str)	返回字节长度 (utf8 一个汉字 3字节)	SELECT LENGTH('abc'); → 3
CHAR_LENGTH(str) 或 LENGTH()	返回字符长度	SELECT CHAR_LENGTH('你好'); → 2
CONCAT(s1, s2, ...)	字符串拼接	SELECT CONCAT('My', 'SQL'); → MySQL
CONCAT_WS(sep, s1, s2, ...)	用分隔符拼接	SELECT CONCAT_WS('-', '2025', '10', '20'); → 2025-10-20
UPPER(str) / UCASE(str)	转大写	SELECT UPPER('mysql'); → MYSQL
LOWER(str) / LCASE(str)	转小写	SELECT LOWER('MYSQL'); → mysql
SUBSTRING(str, pos, len)	截取字符串	SELECT SUBSTRING('abcdef', 2, 3); → bcd
LEFT(str, len)	从左取	SELECT LEFT('abcdef', 3); → abc
RIGHT(str, len)	从右取	SELECT RIGHT('abcdef', 3); → def
LPAD(str, len, pad)	左填充	SELECT LPAD('5', 3, '0'); → 005
RPAD(str, len, pad)	右填充	SELECT RPAD('5', 3, '0'); → 500
TRIM(str)	去首尾空格	SELECT TRIM(' hello '); → hello
REPLACE(str, from_str, to_str)	替换字符串	SELECT REPLACE('abcabc','a','A'); → AbcAbc
INSTR(str, substr)	返回子串首次出现位置	SELECT INSTR('hello','e'); → 2
REVERSE(str)	反转字符串	SELECT REVERSE('abc'); → cba

2. 数学函数 (Numeric Functions)

这些函数用于执行数学计算。

函数名	说明	示例
ABS(x)	返回绝对值	SELECT ABS(-10); → 10
CEIL(x) 或 CEILING(x)	向上取整	SELECT CEIL(3.2); → 4
FLOOR(x)	向下取整	SELECT FLOOR(3.8); → 3
ROUND(x, d)	四舍五入，保留 d 位小数	SELECT ROUND(3.14159, 2); → 3.14
TRUNCATE(x, d)	截断小数位	SELECT TRUNCATE(3.14159, 2); → 3.14
MOD(x, y) 或 x % y	取模	SELECT MOD(10, 3); → 1
POW(x, y) 或 POWER(x, y)	幂运算	SELECT POW(2, 3); → 8
SQRT(x)	平方根	SELECT SQRT(9); → 3
RAND()	返回 0~1 的随机 数	SELECT RAND(); → 0.1234
SIGN(x)	返回符号 (-1, 0, 1)	SELECT SIGN(-3); → -1

3. 日期和时间函数 (Date and Time Functions)

这些函数用于处理日期和时间数据。

函数名	说明	示例
NOW() / CURRENT_TIM ESTAMP()	当前日期时间	2025/10/20 7:30
CURDATE()	当前日期	2025/10/20
CURTIME()	当前时间	7:30:00
DATE(expr)	取日期部分	SELECT DATE('2025-10- 20 08:10:22'); → 2025-10-20
TIME(expr)	取时间部分	SELECT TIME('2025-10- 20 08:10:22'); → 08:10:22

YEAR(date)	返回年份	SELECT YEAR('2025-10-20'); → 2025
MONTH(date)	返回月份	SELECT MONTH('2025-10-20'); → 10
DAY(date)	返回日	SELECT DAY('2025-10-20'); → 20
HOUR(time) / MINUTE(time) / SECOND(time)	分别返回时分秒	SELECT HOUR('08:10:22'); → 8
DATE_ADD(date, INTERVAL n unit)	日期加法	SELECT DATE_ADD('2025-10-20', INTERVAL 5 DAY); → 2025-10-25
DATE_SUB(date, INTERVAL n unit)	日期减法	SELECT DATE_SUB('2025-10-20', INTERVAL 1 MONTH); → 2025-09-20
DATEDIFF(d1, d2)	返回天数差	SELECT DATEDIFF('2025-10-20','2025-10-10'); → 10
TIMEDIFF(t1, t2)	返回时间差	SELECT TIMEDIFF('10:00:00','08:30:00'); → 01:30:00
LAST_DAY(date)	返回该月最后一天	SELECT LAST_DAY('2025-10-20'); → 2025-10-31
DAYOFWEEK(date)	返回星期 (1=周日)	SELECT DAYOFWEEK('2025-10-20'); → 2

4. 聚合函数 (Aggregate Functions)

这些函数对一组值执行计算，并返回单个摘要值，常用于 `GROUP BY` 子句。

函数名	说明	示例
COUNT(expr)	计数	SELECT COUNT(*) FROM employees;
SUM(expr)	求和	SELECT SUM(salary) FROM employees;
AVG(expr)	平均值	SELECT AVG(salary) FROM employees;
MAX(expr)	最大值	SELECT MAX(salary) FROM employees;
MIN(expr)	最小值	SELECT MIN(salary) FROM employees;
GROUP_CONCAT(expr)	分组拼接	SELECT GROUP_CONCAT(name) FROM students; → 'Tom,Jack,Lily'
MONTH(date)	返回月份	SELECT MONTH('2025-10-20'); → 10
DAY(date)	返回日	SELECT DAY('2025-10-20'); → 20
HOUR(time) / MINUTE(time) / SECOND(time)	分别返回时分秒	SELECT HOUR('08:10:22'); → 8
DATE_ADD(date, INTERVAL n unit)	日期加法	SELECT DATE_ADD('2025-10-20', INTERVAL 5 DAY); → 2025-10-25
DATE_SUB(date, INTERVAL n unit)	日期减法	SELECT DATE_SUB('2025-10-20', INTERVAL 1 MONTH); → 2025-09-20
DATEDIFF(d1, d2)	返回天数差	SELECT DATEDIFF('2025-10-20','2025-

TIME, TIME		10-10'); → 10
TIMEDIFF(t1, t2)	返回时间差	SELECT TIMEDIFF('10:00:00','08:30:00'); → 01:30:00
LAST_DAY(date)	返回该月最后一天	SELECT LAST_DAY('2025-10-20'); → 2025-10-31
DAYOFWEEK(date)	返回星期 (1=周日)	SELECT DAYOFWEEK('2025-10-20'); → 2

流程控制函数 (Flow Control Functions)

这些函数提供条件逻辑。

函数名	说明	示例
IF(expr, true_val, false_val)	条件判断	SELECT IF(1>0,'yes','no'); → yes
IFNULL(expr1, expr2)	NULL 替换	SELECT IFNULL(NULL,'default'); → default
NULLIF(expr1, expr2)	若相等则返回 NULL	SELECT NULLIF(5,5); → NULL
CASE WHEN ... THEN ... ELSE ... END	多条件判断	SELECT CASE WHEN score>=90 THEN 'A' WHEN score>=60 THEN 'B' ELSE 'C' END;

系统与信息函数

函数名	说明	示例
VERSION()	MySQL 版本	SELECT VERSION();
USER() / CURRENT_USER()	当前用户	SELECT USER();
DATABASE()	当前数据库名	SELECT DATABASE();
CHARSET(str)	字符集	SELECT CHARSET('abc') ; → utf8mb4
COLLATION(str)	校对规则	SELECT COLLATION('abc') ; → utf8mb4_general_ci

加密函数

函数名	说明	示例
MD5(str)	生成 32 位 MD5 哈希	SELECT MD5('123456'); → e10adc3949ba59abbe56e057f20f883e
SHA1(str) / SHA2(str, 256)	SHA 加密	SELECT SHA2('abc',256) ;
PASSWORD(str)	MySQL 内部加密函数（不推荐手动使用）	SELECT PASSWORD('root');

select 小结

语法结构

代码块

```
1  SELECT [DISTINCT] <字段列表>
2  FROM <表名或子查询>
3  [WHERE <条件>]
4  [GROUP BY <分组字段>]
5  [HAVING <分组过滤条件>]
6  [ORDER BY <排序字段> [ASC|DESC]]
7  [LIMIT <offset>, <行数>];
```

事务

一、什么是事务（Transaction）

事务是一个由一系列数据库操作组成的**逻辑工作单元（Unit of Work）**，
这些操作要么全部成功，要么全部失败回滚。

换句话说：

事务保证数据库在并发访问或系统异常时仍然保持一致性。

二、ACID 四大特性

原则	英文名称	含义	作用和重要性
A	原子性 (Atomicity)	事务是一个 不可分割 的工作单元。	事务中的所有操作要么 全部成功 提交，要么 全部失败 回滚到事务开始前的状态。不存在只完成一部分的情况。
C	一致性 (Consistency)	事务执行前后，数据库从一个 合法状态 转移到另一个 合法状态 。	确保事务执行的结果符合所有的预定义规则（如完整性约束、触发器等）。如果一个事务破坏了数据库的完整性，则会被回滚。
I	隔离性 (Isolation)	并发执行的事务之间 互不干扰 。	确保一个事务在未提交（完成）之前，它所做的修改对其他事务是 不可见 的。隔离性防止了脏读、不可重复读和幻读等并发问题。
D	持久性 (Durability)	一旦事务 提交成功 ，它对数据库中数据的改变就是 永久性 的。	即使系统发生故障（如断电），已提交的修改也不会丢失。通常通过写入 日志文件 和 将数据写入磁盘 来保证。

隔离性不足会导致以下问题：

问题	描述	避免该问题的最低隔离级别
脏读	读到 未提交 事务的数据，该数据随后可能被回滚。	读已提交 (Read Committed)
不可重复读	同一事务内，两次读取同一行数据，发现 数据值被修改 。	可重复读 (Repeatable Read)
幻读	同一事务内，两次执行相同的范围查询，发现 行数发生变化 （有新增或删除）。	串行化 (Serializable)

测试事务

一个转账事务

代码块

```
1  SET autocommit=0; -- 关闭自动提交，有点类似于锁
2  START TRANSACTION ; -- 开启事务
3  UPDATE `account` SET money=money-500 WHERE `name`='A';
4  UPDATE `account` SET money=money+500 WHERE `name`='B';
5  COMMIT; -- 提交事务
6  ROLLBACK; -- 失败则回滚
7  SET autocommit=1;
```

SQL 索引（B+树结构）

SQL 索引可以从**物理结构**和**逻辑功能**两个维度进行分类。

维度一：按物理结构（存储方式）分类

这是索引最核心的分类，主要取决于索引和数据是分离还是捆绑存储。

1. 聚簇索引 (Clustered Index)

- 定义：数据行本身就是按照索引的顺序**物理地**存储在磁盘上。

- **特性：** 一张表**只能有一个**聚簇索引。它的叶子节点存储的就是**整行数据**。
- **用途：** ****主键（Primary Key）****通常就是聚簇索引。它决定了数据在表中的组织方式。
- **示例：**
 - 在您的 `student` 表中，`id` () 列就是聚簇索引。
 - 当你查询 `WHERE id = 1003` 时，数据库直接通过索引找到该 ID 对应的数据物理块，**一步到位**。

2. 非聚簇索引 (Non-Clustered Index)

- **定义：** 索引和数据是**分离**存储的。索引结构存储了列值，但叶子节点存储的不是数据本身，而是指向数据行位置的**指针**（通常是主键值）。
 - **特性：** 一张表可以有**多个**非聚簇索引。
 - **用途：** 用于加速非主键字段的查询。
 - **示例：**
 - 如果您为 `student` 表的 `email` 列创建索引 `idx_email`。
 - 查询 `WHERE email = 'xxx@qq.com'` 时：
 - i. 先在 `idx_email` 索引树中查找到 `email` 对应的 `id`。
 - ii. 使用这个 `id` **回表查询（Look up）**，通过聚簇索引（主键）找到完整的行数据。
-

维度二：按逻辑功能分类

这些索引基于其对列值的约束和用途来命名，它们通常都是**非聚簇索引**（主键除外）。

3. 主键索引 (Primary Key Index)

- **定义：** 一种特殊的索引，用于唯一标识表中的每一行。
- **特性：** 值必须**唯一**且不能为 **NULL**。
- **示例：**
 - `student.id` 和 `subject.No` 都是主键索引。
 - ```
CREATE TABLE student (...) PRIMARY KEY (id);
```

#### 4. 唯一索引 (Unique Index)

- **定义：** 保证索引列的值是唯一的。
- **特性：** 允许包含 **NULL 值**（但通常只允许一个 NULL，具体取决于数据库）。
- **用途：** 用于确保业务规则上的唯一性，例如邮箱或身份证号。
- **示例：**
  - 如果您想确保 `student` 表中的 `email` 不重复。
  - ```
CREATE UNIQUE INDEX uk_email ON student (email);
```

5. 普通索引 (Normal Index)

- **定义：** 最基本的索引，没有任何约束限制。
- **特性：** 允许包含**重复值**和 **NULL 值**。
- **用途：** 加速查询，适用于经常出现在 `WHERE` 子句中的非唯一列。
- **示例：**
 - 为您 `result` 表中的 `score`（成绩）列创建索引，因为分数会有重复。
 - ```
CREATE INDEX idx_score ON result (score);
```

#### 6. 复合索引 (Composite Index)

- **定义：** 在表的**多个列**上创建的索引。
- **特性：** 遵循**最左前缀原则**。查询条件必须从索引的最左侧列开始匹配，索引才能生效。
- **用途：** 针对多条件联合查询进行优化。
- **示例：**
  - 为 `student` 表创建复合索引：`idx_addr_bday (address, birthday)`。
  - 查询 `WHERE address = '北京朝阳' AND birthday = '2025-01-01'` 时，索引**完全生效**。
  - 查询 `WHERE address = '北京朝阳'` 时，索引**部分生效**（使用了最左前缀）。
  - 查询 `WHERE birthday = '2025-01-01'` 时，索引**失效**（没有使用最左前缀）。

## 7. 全文索引 (Full-Text Index)

- **定义：** 专门用于在大量文本数据中进行**关键词检索**。
- **特性：** 搜索的是单词或词组，而不是基于值的精确比较或前缀匹配。需要使用特定的 `MATCH...AGAINST` 语法。
- **用途：** 用于提高 `address` 字段中模糊搜索的效率。
- **示例：**
  - `CREATE FULLTEXT INDEX fts_address ON student (address);`
  - 查询： `SELECT * FROM student WHERE MATCH (address) AGAINST ('北京');`

## 关键优化概念：覆盖索引 (Covering Index)

这是性能优化的一个高级技巧，与索引类型无关，但与**查询方式**和**索引内容**有关。

- **定义：** 当非聚簇索引的叶子节点中已经包含了查询所需的所有列时，查询可以直接从索引中获取结果，**无需执行“回表”操作**。
- **示例：**
  - 我们为 `result` 表创建复合索引： `idx_id_score (id, score)`。
  - 查询： `SELECT id, score FROM result WHERE id = 1001 AND score > 80;`
  - **效果：** 数据库在 `idx_id_score` 索引树中就能找到所有需要的 `id` 和 `score` 值，避免了昂贵的磁盘 I/O 操作，查询效率极高。

## 测试100万条数据索引

插入100万数据

代码块

```
1 DELIMITER $$
2 CREATE PROCEDURE insert_data()
3 BEGIN
4 DECLARE i INT DEFAULT 0;
5 DECLARE batch_size INT DEFAULT 10000; -- 每批次1万条
6 DECLARE sql_text TEXT DEFAULT '';
7
8 WHILE i < 1000000 DO
```

```

9 SET sql_text = 'INSERT INTO `user`(`name`,`phone`,`password`,`age`)
VALUES';
10
11 -- 拼接一批INSERT语句
12 WHILE i < 1000000 AND MOD(i, batch_size) <> 0 DO
13 SET sql_text = CONCAT(sql_text,
14 '("用户', i, '",',
15 '1', FLOOR(2000000000 + RAND() * (999999999 - 2000000000 +
16 1)), '",',
17 '""', UUID(), '",',
18 FLOOR(10 + RAND() * (100 - 10 + 1)), '),'
19);
20 SET i = i + 1;
21 END WHILE;
22
23 -- 去掉最后一个逗号
24 SET sql_text = LEFT(sql_text, LENGTH(sql_text) - 1);
25 SET @s = sql_text;
26 PREPARE stmt FROM @s;
27 EXECUTE stmt;
28 DEALLOCATE PREPARE stmt;
29 END WHILE;
30 END $$
31
32 DELIMITER ;
33 SET autocommit = 0;
34 CALL insert_data();
35 COMMIT;

```

没有索引时下列查询用了4秒左右，

建立索引后，仅用0.002秒

代码块

```

1 SELECT `id` FROM `user` WHERE `name`='用户99999';
2 CREATE INDEX idx_age ON `user`(`age`);

```

## 数据库用户管理

### 一.用户结构

'用户名'@'主机名'

示例：

`'root'@'localhost'`：只允许本机登录

- ``'admin'@'%': 允许从任意主机登录

- ``'dev'@'192.168.1.%': 仅允许局域网特定 IP 登录

## 二.创建用户

语法:

``sql

```
CREATE USER '用户名'@'主机' IDENTIFIED BY '密码';
```

示例:

```
CREATE USER 'dev'@'localhost' IDENTIFIED BY '123456';
```

```
CREATE USER 'web'@'%' IDENTIFIED BY 'web_pass';
```

---

## 三、查看用户

查看所有用户:

```
SELECT user, host FROM mysql.user;
```

查看当前登录用户:

```
SELECT USER(), CURRENT_USER();
```

---

## 四、授权 (GRANT)

语法:

```
GRANT 权限列表 ON 数据库.表 TO '用户名'@'主机';
```

示例:

```
GRANT SELECT, INSERT, UPDATE, DELETE ON mydb.* TO 'dev'@'localhost';
```

```
GRANT ALL PRIVILEGES ON mydb.* TO 'admin'@'%';
```

```
GRANT ALL PRIVILEGES ON *.* TO 'root'@'%' WITH GRANT OPTION;
```

刷新权限:

```
FLUSH PRIVILEGES;
```

---

## 五、撤销权限

语法:

```
REVOKE 权限列表 ON 数据库.表 FROM '用户名'@'主机';
```

示例：

```
REVOKE INSERT, UPDATE ON mydb.* FROM 'dev'@'localhost';
```

## 六、修改用户信息

修改密码：

```
ALTER USER 'dev'@'localhost' IDENTIFIED BY 'newpassword';
```

旧版 MySQL：

```
SET PASSWORD FOR 'dev'@'localhost' = PASSWORD('newpassword');
```

修改用户名：

```
RENAME USER 'dev'@'localhost' TO 'developer'@'localhost';
```

## 七、删除用户

语法：

```
DROP USER '用户名'@'主机';
```

示例：

```
DROP USER 'web'@'%';
```

## 八、常见权限说明

| 权限名            | 说明       |
|----------------|----------|
| SELECT         | 查询数据     |
| INSERT         | 插入数据     |
| UPDATE         | 更新数据     |
| DELETE         | 删除数据     |
| CREATE         | 创建数据库或表  |
| DROP           | 删除数据库或表  |
| ALTER          | 修改表结构    |
| INDEX          | 创建或删除索引  |
| GRANT OPTION   | 可将权限授予他人 |
| ALL PRIVILEGES | 拥有全部权限   |

## 九、常用管理命令

查看用户权限：

```
SHOW GRANTS FOR 'dev'@'localhost';
```

刷新权限缓存：

```
FLUSH PRIVILEGES;
```

仅允许本地 root 登录：

```
DROP USER 'root'@'%';
```

## 十、安全管理建议

1. 禁止使用空密码用户。
2. 授权时遵循最小权限原则。
3. 定期修改密码。
4. 删除不再使用的账户。
5. 不给普通应用账户授予 `GRANT OPTION` 或 `SUPER` 权限。

## mysql备份

逻辑备份： `mysqldump`

`mysqldump` 是 MySQL 自带的最常用的逻辑备份工具。它将数据库内容导出为一个包含 `CREATE TABLE` 和 `INSERT INTO` 语句的 `.sql` 文件。

### 1. 常用命令示例

**基本语法：** `mysqldump -u [用户名] -p[密码] [选项] [数据库名] [表名] > [备份文件名.sql]`

**注意：** `-p` 和密码之间**没有空格**。

### a. 备份单个数据库

代码块

```
1 mysqldump -u root -p'your_password' student > student_backup.sql
```

- 这将备份 `student` 数据库的完整结构和数据到 `student_backup.sql` 文件。

### b. 备份所有数据库

```
代码块mysqldump -u root -p'your_password' --all-databases > all_databases_backup.sql
```

### c. 备份特定表

代码块

```
1 mysqldump -u root -p'your_password' student result subject > student_tables_backup.sql
```

- 备份 `student` 数据库中的 `result` 和 `subject` 表。

### d. 仅备份结构（不包含数据）

代码块

```
1 mysqldump -u root -p'your_password' --no-data student > student_schema_only.sql
```

### e. 备份时包含存储过程和函数

为了确保存储过程、函数和触发器也被备份，需要添加 `--routines` 和 `--triggers` 选项。

代码块

```
1 mysqldump -u root -p'your_password' --all-databases --routines --triggers > full_backup.sql
```

## 2. 如何恢复

使用 `mysql` 客户端工具可以轻松地从 `.sql` 文件恢复数据。

代码块

```
1 # 1. 登录 MySQL
2 mysql -u root -p'your_password'
3
4 # 2. 创建一个空的数据库（如果需要）
5 CREATE DATABASE student;
6
7 # 3. 切换到该数据库
8 USE student;
9
10 # 4. 从 .sql 文件导入数据
11 SOURCE /path/to/your/student_backup.sql;
```

或者直接在 shell 中使用重定向：

代码块

```
1 mysql -u root -p'your_password' student < student_backup.sql
```

## 物理备份：Percona XtraBackup

对于大型数据库（几十GB到TB级别），`mysqldump` 的速度太慢。物理备份是更好的选择。Percona XtraBackup 是一个广泛使用的开源热备份工具。

### 3. 工作原理

XtraBackup 在不锁定数据库的情况下，复制 InnoDB 数据文件。它通过记录复制过程中的变更（redo log），确保备份数据的一致性。

### 4. 备份与恢复流程（概念）

物理备份的过程比逻辑备份复杂，通常分为三步：

#### 1. 创建备份 (Backup):

- 使用 `xtrabackup` 命令创建一个完整的物理备份副本。这个过程对正在运行的数据库影响很小。

#### 2. 准备备份 (Prepare):

- 备份创建后，数据文件可能处于不一致的状态。需要使用 `xtrabackup --prepare` 命令，将备份期间产生的事务日志应用到数据文件中，使其达到一个一致性的状态。

#### 3. 恢复备份 (Restore):

- 停止 MySQL 服务，将准备好的备份文件复制回 MySQL 的数据目录，并修正文件权限，然后重启 MySQL 服务。

物理备份速度极快，是大型生产环境的首选方案。

## 增量备份与时间点恢复 (Point-in-Time Recovery)

除了全量备份，我们还需要更精细的恢复策略。

### 4. 二进制日志 (Binary Log)

- **是什么：** Binlog 是 MySQL 的一个日志文件，它记录了所有导致数据变更的 SQL 语句（`INSERT`，`UPDATE`，`DELETE` 等）。
- **用途：** Binlog 是实现**增量备份**和**时间点恢复**的关键。

### 5. 时间点恢复策略

假设您在凌晨 2:00 进行了全量备份，但在上午 10:30 发生了误删除操作。恢复步骤如下：

1. **恢复全量备份：** 将数据库恢复到凌晨 2:00 的状态。
2. **应用二进制日志：** 使用 `mysqlbinlog` 工具，将从凌晨 2:00 到上午 10:29（误操作发生前）的所有 binlog 应用到数据库中。

这样，您就可以将数据精确恢复到事故发生前的任意一个时间点。

## 数据库三大范式 (3NF)

三大范式是设计规范表结构，**减少冗余**和**保证数据一致性**的准则。

| 范式  | 核心要求 (做什么)                            | 解决的问题                           | 示例：拆分哪个字段                                                     |
|-----|---------------------------------------|---------------------------------|---------------------------------------------------------------|
| 1NF | <b>原子性：</b> 列不可再分。                    | 避免在单列中存储多个独立信息。                 | 联系方式 (电话+邮箱) \$\to\$ phone 和 email 两列                         |
| 2NF | <b>完全依赖：</b> 所有非主键列必须依赖 <b>整个主键</b> 。 | 消除对 <b>复合主键</b> 的 <b>部分依赖</b> 。 | result 表中，subjectName 仅依赖于 subjectNo（部分），应该移到独立的 subject 表。   |
| 3NF | <b>消除传递依赖：</b> 非主键列之间不能有依赖关系。         | 消除非主键列依赖于 <b>另一个非主键列</b> 的情况。   | student 表中，headTeacher 依赖于 classNo（非主键），应该将班级信息移到独立的 class 表。 |

## JDBC 详解

JDBC (Java Database Connectivity) 是 Sun Microsystems（现属于 Oracle）推出的一套用于执行 SQL 语句的 **Java API 规范**。它定义了一系列接口和类，使得 Java 程序可以独立于具体的数据库产品（如 MySQL、Oracle、PostgreSQL 等）进行操作。

### 一、JDBC 的核心工作原理：驱动管理器

JDBC 能够实现“一次编写，处处运行”的关键在于其采用的**驱动管理器 (Driver Manager)** 架构。

1. **JDBC API (接口层):** 位于 Java 应用程序和驱动程序之间。它定义了 Java 程序与数据库交互的标准方法（如 `Connection`，`Statement`，`ResultSet` 等接口）。
2. **JDBC Driver (驱动程序):** 这是由各数据库厂商（如 MySQL, Oracle）提供的**具体实现**，用于将 JDBC 接口的调用翻译成数据库能理解的通信协议。
3. **JDBC Driver Manager (驱动管理器):** 负责加载和管理驱动程序，并根据应用程序提供的连接信息（URL）来选择和建立合适的数据库连接。

## 二、JDBC 编程的六个基本步骤

无论是执行简单的查询还是复杂的事务操作，标准的 JDBC 流程都遵循以下六个步骤（通常被称为“六部曲”）：

### 1. 注册驱动 (Load the Driver)

在 JDBC 4.0 之后，通常不需要手动调用 `Class.forName()`，但为了兼容性或明确性，仍可以这样写。

代码块

```
1 // 示例：注册 MySQL 驱动
2 Class.forName("com.mysql.cj.jdbc.Driver");
```

### 2. 建立连接 (Establish Connection)

使用 `DriverManager.getConnection()` 方法，通过数据库连接 URL、用户名和密码来建立与数据库的连接。

代码块

```
1 String url="jdbc:mysql://localhost:3306/school?serverTimezone=UTC";
2 String user = "root";
3 String password = "mypassword";
4 Connection connection = DriverManager.getConnection(url, user, password);
```

### 3. 创建语句对象 (Create Statement)

连接对象 (Connection) 用于创建不同类型的语句对象，用于发送 SQL。

代码块

```
1 // 用于发送简单的、无参数的 SQL 语句
2 Statement statement = connection.createStatement();
```

### 4. 执行 SQL 语句 (Execute SQL)

根据 SQL 语句的类型，使用不同的执行方法：

| SQL 类型                    | JDBC 方法                | 返回值             | 示例                                                                            |
|---------------------------|------------------------|-----------------|-------------------------------------------------------------------------------|
| 查询 (SELECT)               | executeQuery()<br>( )  | ResultSet (结果集) | ResultSet rs = statement.executeQuery("SELECT * FROM student");               |
| 更新 (INSERT/UPDATE/DELETE) | executeUpdate()<br>( ) | int (受影响的行数)    | int rows = statement.executeUpdate("UPDATE student SET score=90 WHERE id=1"); |
| DDL (CREATE/DROP)         | execute()<br>( )       | boolean         | boolean success = statement.execute("CREATE TABLE temp (...");                |

5. 处理结果 (Process Results)

如果是查询操作，使用 `ResultSet` 对象遍历和提取数据。

代码块

```
1 //查看查询结果
2 while(rs.next())
3 {
4 //知道具体列的属性，则可用，getInt(),getString()等
5 System.out.println(rs.getObject("id"));
6 System.out.println(rs.getObject("name"));
7 System.out.println(rs.getObject("birthday"));
8 System.out.println(rs.getObject("password"));
9 System.out.println(rs.getObject("email"));
10 System.out.println(rs.getObject("address"));
11 }
12 }
```

6. 关闭资源 (Close Resources)

为了释放系统资源，必须按照**先开后关**的原则，关闭 `ResultSet`、`Statement` 和 `Connection`。在 Java 7 之后，通常推荐使用 **try-with-resources** 语法来确保资源被自动关闭。

代码块

```
1 // try-with-resources 示例，确保 rs, stmt, conn 都会自动关闭
```

```

2 try (Connection conn = DriverManager.getConnection(url, user, password);
3 Statement stmt = conn.createStatement();
4 ResultSet rs = stmt.executeQuery("SELECT * FROM student")) {
5
6 // ... 处理结果
7
8 } catch (SQLException e) {
9 e.printStackTrace();
10 }

```

### 三、三种主要的 Statement 对象

为了处理不同类型的 SQL 语句和提高性能，JDBC 提供了三种主要的语句对象：

#### 1. Statement：

- **特点：** 用于执行简单的、静态的 SQL 语句。每次执行都会重新编译 SQL。
- **缺点：** 容易受到 **SQL 注入攻击**。

#### 2. PreparedStatement (预编译语句):

- **特点：** **推荐使用**。用于执行带参数的动态 SQL 语句。
- **原理：** SQL 语句只编译一次，后续执行时只需传入参数。
- **优点：** 性能高，**自动防止 SQL 注入**（因为参数值会被单独处理，而不是拼接到 SQL 字符串中）。

代码块

```

1 PreparedStatement ps = conn.prepareStatement("UPDATE student SET name = ?
2 WHERE id = ?");
3 ps.setString(1, "新的名字"); // 设置第一个参数
4 ps.setInt(2, 1001); // 设置第二个参数
5 ps.executeUpdate();

```

#### 3. CallableStatement (可调用语句):

- **特点：** 用于调用数据库中预定义的**存储过程 (Stored Procedures)** 和**函数 (Functions)**。

### JDBC 事务

代码块

```

1 import java.sql.*;
2 public class TestTransaction {
3 public static void main(String[] args) throws ClassNotFoundException,
4 SQLException {

```

```

4 //1.注册驱动
5 Class.forName("com.mysql.cj.jdbc.Driver");
6 //2.用户信息和url
7 String url = "jdbc:mysql://localhost:3306/shop?serverTimezone=UTC";
8 String user = "root";
9 String password = "100018";
10 //3.建立连接
11 try
12 {
13 Connection connection = DriverManager.getConnection(url, user,
password);
14 String sql1="update account set money=money-100 where name=?";
15 String sql2="update account set money=money+100 where name=?";
16 PreparedStatement pst=null;
17 try{
18 connection.setAutoCommit(false);
19 pst=connection.prepareStatement(sql1);
20 pst.setString(1,"B");
21 pst.executeUpdate();
22 pst=connection.prepareStatement(sql2);
23 pst.setString(1,"A");
24 pst.executeUpdate();
25 connection.commit();
26 }catch(SQLException e)
27 {
28 connection.rollback();
29 e.printStackTrace();
30 }
31 }
32 catch (SQLException e) {
33 e.printStackTrace();
34 }
35
36 }
37 }

```

## SQL 注入

SQL 注入通常发生在应用程序使用**字符串拼接**的方式来构建 SQL 查询语句时。

### 1. 正常操作 (安全)

假设您的应用程序有一个登录功能，它构造的 SQL 语句是：

SQL

代码块

```
1 SELECT * FROM users WHERE username = 'bob' AND password = 'password123';
```

## 2. 注入攻击 (危险)

攻击者在用户名输入框中输入： `' OR '1'='1'`

应用程序使用字符串拼接后，实际执行的 SQL 语句变为：

SQL

代码块

```
1 SELECT * FROM users WHERE username = '' OR '1'='1' AND password = '...';
```

语句分析：

- `' OR '1'='1'` 这部分会使 `WHERE` 子句永远为真（`'1'='1'` 总是成立）。
- 数据库会返回 **所有** 用户记录，成功绕过了密码验证，登录了第一个用户（通常是管理员）。